

# 6 PORTFOLIO PROJECTS EVERY ME STUDENT NEEDS

---

Build these. Get hired.

Full breakdowns, components, step-by-step guides, and resource links

By @1nska

NYU Shanghai — 2026

# Why Your Portfolio Matters More Than Your GPA

Here's what most ME programs won't tell you: the students landing the best jobs aren't the ones with the highest grades. They're the ones who can point to something they built and say "I made this, and here's how it works."

A strong portfolio does three things: it proves you can take a project from concept to completion, it shows you understand real-world constraints beyond textbook problems, and it gives interviewers something concrete to talk about instead of asking you to explain the second law of thermodynamics for the hundredth time.

These 6 projects cover the key areas ME employers look for: CAD and prototyping, thermal and fluid systems, control systems, manufacturing, instrumentation, and mechatronics. Build 2-3 of these and you'll stand out from 90% of applicants.

**How to use this guide:** Each project includes a component list, step-by-step build instructions, clickable resource links, and a "level up" tip to take it further. Start with whichever excites you most.

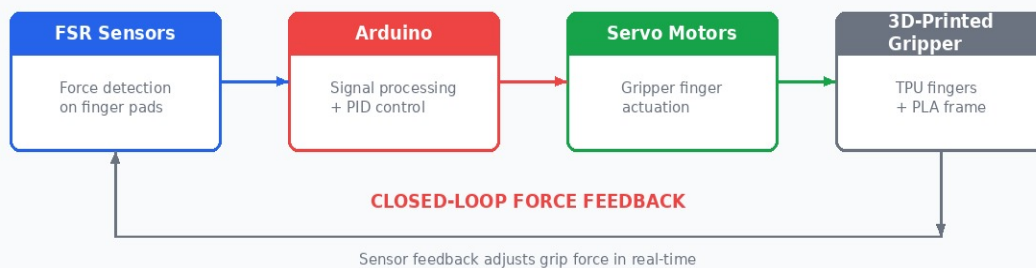
# #1

## 3D-Printed Robotic Gripper with Force Feedback

PROTOTYPING • MECHATRONICS • SENSORS

**Difficulty:** Beginner–Intermediate | **Estimated cost:** \$25–40

### ROBOTIC GRIPPER — SYSTEM DIAGRAM



### Key Components

- **FSR 402 Sensors**  
Detects 0.1N - 10N force range
- **SG90 Micro Servos**  
Low-cost, sufficient torque for gripper
- **TPU Flexible Filament**  
Soft finger pads for better grip
- **Voltage Divider (10K)**  
Converts FSR resistance to voltage
- **Arduino Uno/Nano**

### Wiring Overview

- FSR → A0, A1 (analog input via voltage divider)
- Servo 1 → D9 (PWM output)
- Servo 2 → D10 (PWM output)
- Power: 5V USB or external 5V regulated
- Ground: Common ground for all components

**Why recruiters love this:** It shows you can go from CAD model to functional prototype with real sensor integration. Robotics companies and automation firms specifically look for this end-to-end prototyping skill.

Design and 3D-print a robotic gripper that uses force-sensitive resistors (FSRs) to detect how hard it's gripping and automatically adjusts pressure. It should handle different objects — a soft tomato vs a hard ball — without crushing either one. This is the exact problem Amazon and warehouse automation companies are solving at scale.

### What you need:

- 3D printer access (university lab or online service like Craftcloud)
- Arduino Uno or Nano
- 2x SG90 micro servo motors
- 2x Force-sensitive resistors (FSR 402)
- Flexible filament (TPU) for gripper fingers + PLA for frame

- Jumper wires, breadboard, 5V power supply

## Skills it proves:

- CAD design for manufacturing — designing parts that actually print well
- Sensor integration and calibration
- Closed-loop control — using sensor feedback to adjust actuator output
- Rapid prototyping and iterative design
- Arduino programming with real-time sensor reading

## How to build it:

**Step 1:** Design the gripper in CAD. Use Fusion 360 or SolidWorks — design a two-finger gripper with servo-actuated linkages. Design for 3D printing: avoid overhangs, add chamfers, and use proper clearances for moving joints.

**Step 2:** Print and assemble. Print the frame in PLA and the finger pads in TPU (flexible filament) for better grip. Assemble the linkage mechanism and mount the servos.

**Step 3:** Wire the FSR sensors. Mount force-sensitive resistors on the inner surface of each finger pad. Wire them to the Arduino's analog inputs through a voltage divider circuit (10K resistor).

**Step 4:** Write the control loop. Read FSR values → compare to target force → adjust servo position. Start with a simple proportional controller: if force is too high, open slightly; if too low, close slightly.

**Step 5:** Test with different objects. Calibrate your force thresholds for soft objects (tomato, egg) vs hard objects (ball, cube). Record success/failure rates for your documentation.

**Step 6:** Document everything. Take photos at every stage. Create a GitHub repo with your CAD files (STL + source), Arduino code, wiring diagram, and a demo video showing the gripper handling different objects.

## Useful links & resources:

- [Fusion 360 free for students](#)
- [FSR sensor guide — Adafruit](#)
- [Thingiverse robotic gripper designs](#)
- [Arduino servo tutorial](#)
- [TPU printing guide](#)
- [Seeed Studio reBot Arm B601 — open source 6-DoF arm \(GitHub + full docs\)](#)

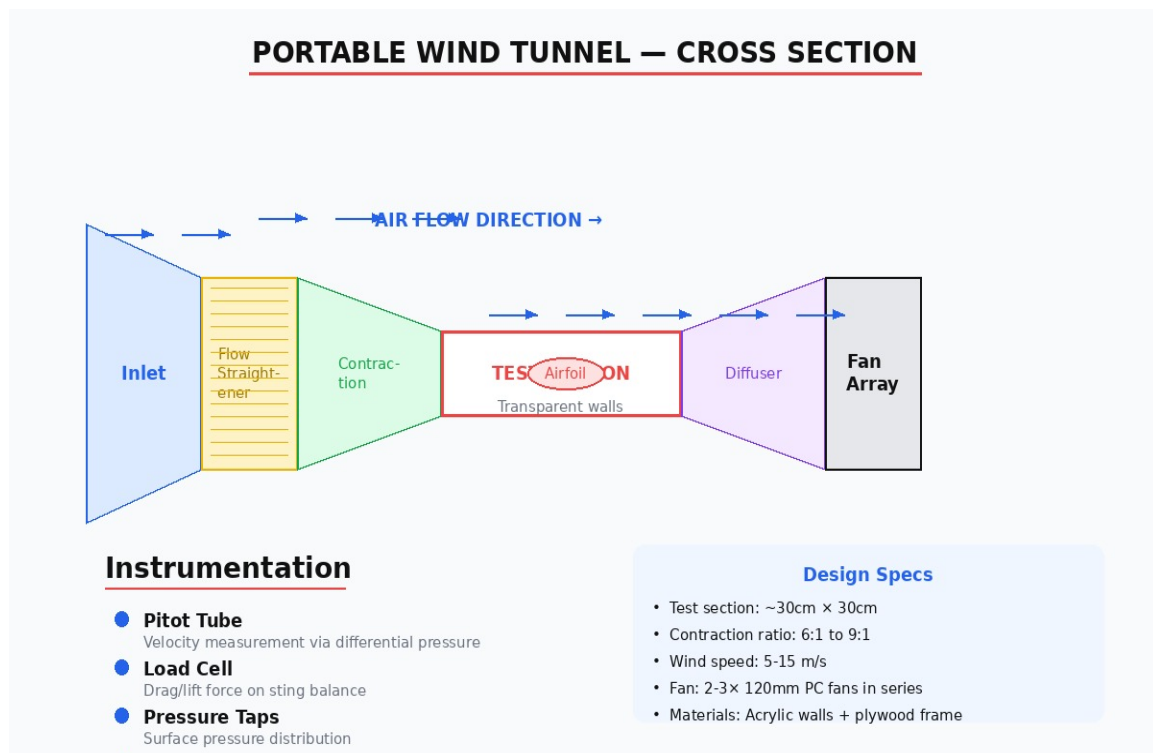
**Level up:** Add computer vision using a Raspberry Pi camera so the gripper detects the object type before gripping and pre-selects the right force profile. If you want to go even further, check out Seeed Studio's open source reBot Arm B601 — a full 6-DoF robotic arm with gripper, sub-\$1K build cost, ROS2 support, and complete documentation. It's the best open source arm project out right now for learning real robotics.

# #2

## Portable Wind Tunnel with Data Acquisition

FLUIDS • AERODYNAMICS • INSTRUMENTATION

Difficulty: Intermediate | Estimated cost: \$40–70



**Why recruiters love this:** Aerodynamics and fluid mechanics are core ME competencies. Building your own test equipment shows you understand both the theory and the practical challenges of measurement — and that you can design experiments, not just run them.

Build a small-scale open-circuit wind tunnel (~30cm test section) with a fan-driven flow, a honeycomb flow straightener, a contraction nozzle, and an instrumented test section. Use pressure sensors and an Arduino to measure velocity profiles and drag forces on test objects. Visualize airflow with smoke or string tufts.

### What you need:

- Acrylic or plywood sheets for tunnel walls
- 120mm PC fans (2-3 in series) or a centrifugal blower
- Drinking straws (honeycomb flow straightener — seriously, it works)
- Arduino Mega + BMP280 or differential pressure sensor (MPXV7002DP)
- Small load cell (for drag force measurement)

- Incense sticks for smoke visualization
- 3D-printed airfoil test models (NACA profiles)

### Skills it proves:

- Fluid mechanics — boundary layers, pressure distributions, Reynolds number
- Experimental design and data acquisition
- Sensor calibration and uncertainty analysis
- CAD and fabrication of precision test equipment
- Data analysis and visualization (Python + matplotlib)

### How to build it:

**Step 1:** Design the tunnel geometry. Follow established open-circuit wind tunnel design rules: contraction ratio of 6:1 to 9:1, test section length  $\sim 1.5\times$  the width. Sketch it in CAD first to verify dimensions.

**Step 2:** Build the structure. Cut acrylic or plywood panels. The test section should be transparent (acrylic) for visualization. Use silicone sealant on joints to prevent air leaks.

**Step 3:** Build the flow straightener. Bundle  $\sim 200$  drinking straws (cut to 10cm) tightly into the settling chamber. This kills turbulence and creates uniform flow. Add a fine mesh screen after the straws.

**Step 4:** Install fans and test airflow. Mount fans at the exit (suction configuration is more uniform than blowing). Measure velocity at multiple points across the test section to verify uniformity. Target 5-15 m/s.

**Step 5:** Set up instrumentation. Wire the differential pressure sensor to a pitot tube for velocity measurement. Mount the load cell on a sting balance to measure drag. Log all data to the Arduino at 100+ Hz.

**Step 6:** Run experiments. 3D-print NACA 0012 and NACA 4412 airfoils. Measure drag at different angles of attack. Compare your data to published results. Use incense smoke for flow visualization and photograph it.

### Useful links & resources:

- [NASA wind tunnel design guide \(PDF\)](#)
- [NACA airfoil coordinate generator](#)
- [MPXV7002DP pressure sensor datasheet](#)
- [Python matplotlib for data visualization](#)
- [Open-source wind tunnel project — MIT](#)

**Level up:** Add a hot-wire anemometer for turbulence intensity measurements, or use PIV (Particle Image Velocimetry) with a laser pointer and camera to visualize velocity fields quantitatively. Publish your results and CAD files as an open-source project.

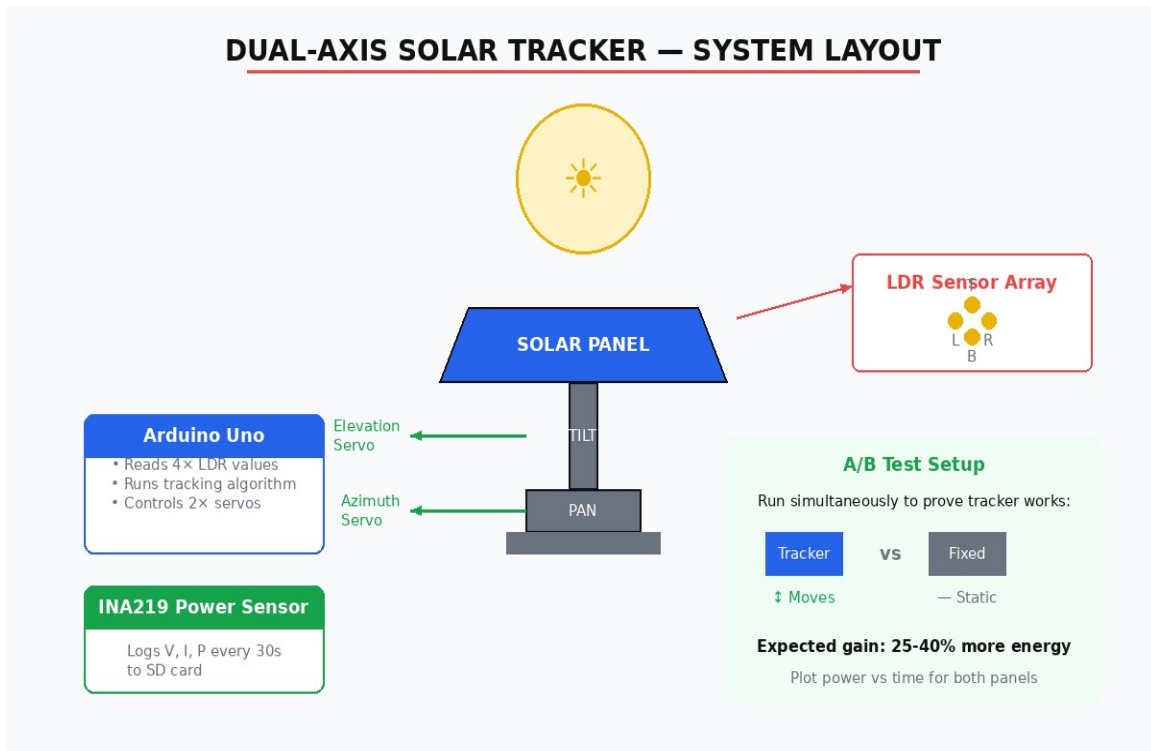


# #3

## Dual-Axis Solar Tracker

CONTROL SYSTEMS • RENEWABLE ENERGY • MECHANICAL DESIGN

Difficulty: Intermediate | Estimated cost: \$35–55



**Why recruiters love this:** Renewable energy is booming and every company in the space needs MEs who understand both the mechanical design and the control systems behind tracking systems. This project proves you can do both.

Build a dual-axis solar tracker that automatically orients a small solar panel to face the sun throughout the day. Use light-dependent resistors (LDRs) to detect the sun's position and servo motors to adjust pan and tilt angles. Log power output data to prove your tracker generates more energy than a fixed panel.

### What you need:

- Small solar panel (5-10W)
- Arduino Uno
- 2x MG996R servo motors (high torque for outdoor use)
- 4x LDR sensors + 10K resistors
- Pan-tilt mounting bracket (3D print or buy)

- INA219 current/voltage sensor for power logging
- Weatherproof enclosure for electronics

### Skills it proves:

- Mechanical design for outdoor/environmental conditions
- Control system design — closed-loop sun tracking
- Sensor calibration and signal conditioning
- Energy systems — quantitative performance analysis
- Weatherproofing and design for reliability

### How to build it:

**Step 1:** Design the mount. Create a pan-tilt mechanism with two degrees of freedom. The base servo controls azimuth (east-west), the top servo controls elevation (up-down). Design for the weight of your panel + wind loads.

**Step 2:** Build the LDR sensor array. Mount 4 LDRs in a cross pattern with a divider wall between them. When the sun is centered, all 4 read equal values. An imbalance tells you which direction to move.

**Step 3:** Write the tracking algorithm. Compare opposite LDR pairs: if left > right, rotate left. If top > bottom, tilt up. Add a dead band ( $\pm 5\%$ ) to prevent hunting/oscillation around the sun position.

**Step 4:** Add power monitoring. Wire an INA219 sensor to measure the solar panel's voltage and current output. Log data to an SD card every 30 seconds — you need this data to prove the tracker works.

**Step 5:** Set up the comparison test. Place an identical fixed panel next to your tracker. Log both panels' output simultaneously over a full day. Calculate the energy gain percentage.

**Step 6:** Analyze and document. Plot power output vs. time for both panels using Python. Calculate total daily kWh for each. Create graphs showing the tracker's advantage. Document the full build on GitHub with CAD files and data.

### Useful links & resources:

- [Solar tracker theory — how tracking increases yield](#)
- [INA219 power sensor tutorial — Adafruit](#)
- [Pan-tilt bracket 3D print files](#)
- [Arduino solar tracker tutorial](#)
- [Servo motor selection guide](#)

**Level up:** Add WiFi logging so you can monitor performance remotely via a web dashboard. Then add a weather station (temperature, wind speed, cloud cover) and correlate environmental conditions with tracking performance. This data set becomes a publishable research project.

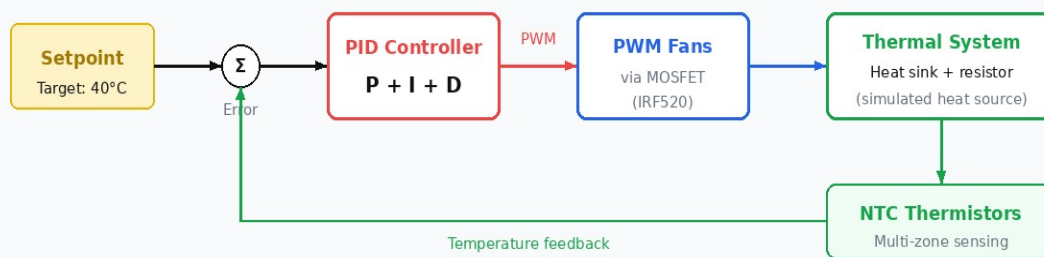
# #4

## PID-Controlled Thermal Management System

HEAT TRANSFER • CONTROL SYSTEMS • EMBEDDED

Difficulty: Intermediate | Estimated cost: \$30–50

### PID THERMAL MANAGEMENT — CONTROL LOOP



#### PID Tuning Guide (Ziegler-Nichols)

1. Set  $I = 0$ ,  $D = 0$ . Increase  $P$  until system oscillates
  2. Note the critical gain ( $K_u$ ) and oscillation period ( $T_u$ )
  3. Calculate:  $P = 0.6 \times K_u$ ,  $I = T_u/2$ ,  $D = T_u/8$
  4. Fine-tune: reduce overshoot with more  $D$
  5. Target: settling time  $< 30s$ , overshoot  $< 10\%$
- Log temp vs. time during tuning — this IS your portfolio data

#### System Specs

- Target accuracy:  $\pm 0.5^\circ\text{C}$  steady state
- Control rate: 10-50 Hz loop
- Zones: 3-4 independent thermistors
- Fans: 2x 12V PWM (Noctua recommended)
- Heat source: 10-50W power resistor
- Display: LCD or OLED for real-time readout
- Data logging: SD card at 1 Hz

**Why recruiters love this:** Thermal management is critical in electronics, automotive, aerospace, and data centers. Building a PID-controlled cooling system shows you understand heat transfer fundamentals AND can implement real control loops — the exact combination companies like Tesla, Apple, and server farm operators need.

Build a smart cooling system that monitors temperature across multiple zones using thermistors, runs a PID control loop to adjust fan speeds via PWM, and maintains a target temperature within  $\pm 0.5^\circ\text{C}$ . Display real-time data on an LCD and log performance data for analysis.

### What you need:

- Arduino Mega
- 3-4x NTC thermistors (10K) for multi-zone sensing
- 2x 12V PWM-controllable fans (Noctua or similar)
- MOSFET modules (IRF520) for fan speed control
- Aluminum heat sink + resistive heater (simulated heat source)

- 16x2 LCD display or OLED (SSD1306)
- 12V power supply + buck converter for Arduino

### Skills it proves:

- Heat transfer fundamentals applied in practice
- PID control — tuning, implementation, and performance analysis
- Multi-zone thermal sensing and management
- PWM motor control through MOSFETs
- Data logging and system characterization

### How to build it:

**Step 1:** Build the thermal test bed. Mount a power resistor (10-50W) on an aluminum heat sink as your heat source. Attach thermistors at different distances from the source to create temperature zones.

**Step 2:** Set up fan control. Wire fans through IRF520 MOSFETs connected to Arduino PWM pins. Verify you can control fan speed from 0-100% smoothly. Measure airflow at different PWM values.

**Step 3:** Implement the PID controller. Start with proportional only (P). Set a target temperature (e.g., 40°C). Gradually add derivative (D) to reduce overshoot, then integral (I) to eliminate steady-state error.

**Step 4:** Tune the PID. Use the Ziegler-Nichols method: increase P until oscillation, note the period, then calculate P, I, D values. Fine-tune from there. Log temperature vs. time to show your tuning process.

**Step 5:** Add multi-zone control. Implement independent PID loops for each zone, or a cascaded controller where the hottest zone drives the primary response. Show the system maintaining all zones within  $\pm 0.5^\circ\text{C}$ .

**Step 6:** Run step-response tests. Apply sudden heat loads and record how the system responds. Plot temperature vs. time, fan speed vs. time, and calculate settling time, overshoot, and steady-state error. This data is your portfolio showpiece.

### Useful links & resources:

- [PID controller explained — Brian Douglas \(YouTube\)](#)
- [Ziegler-Nichols tuning method](#)
- [Arduino PID library](#)
- [NTC thermistor guide — Adafruit](#)
- [PWM fan control with MOSFET tutorial](#)

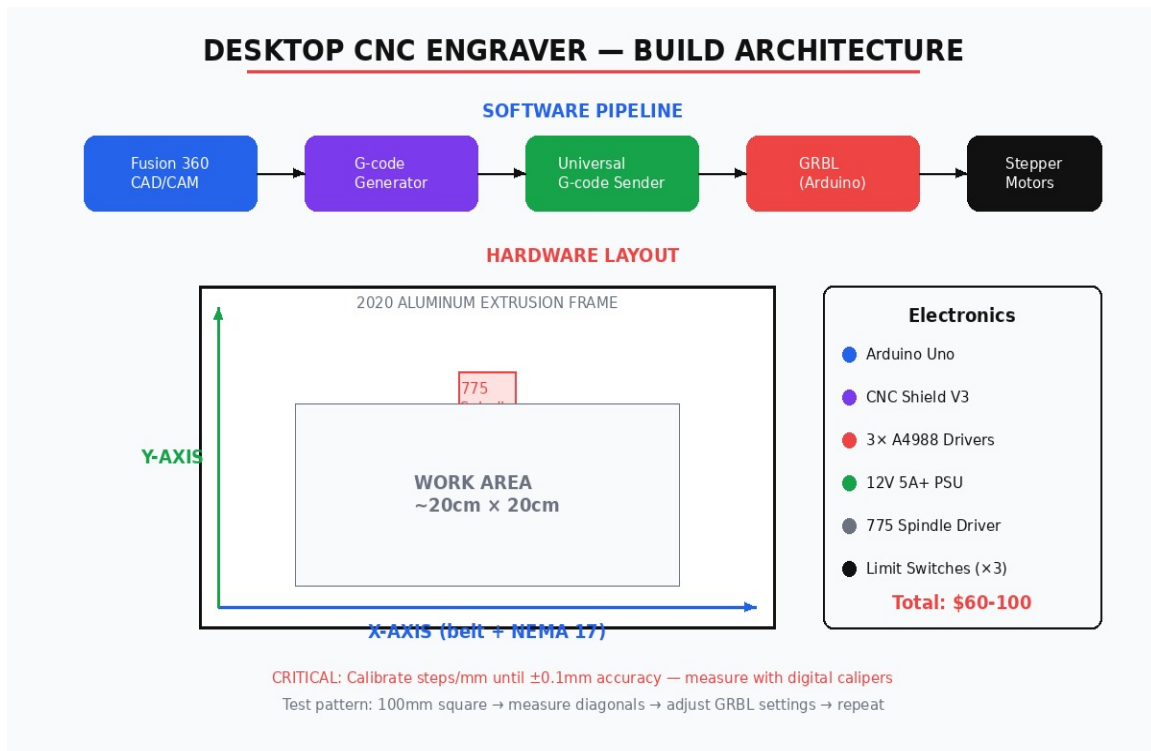
**Level up:** Replace the simple heat source with a real CPU or GPU under load. Benchmark thermal performance against the stock cooler. If your PID system maintains lower temps at the same noise level, you've built something companies would actually ship.

# #5

## Desktop CNC Engraver (Built From Scratch)

MANUFACTURING • CNC • MECHANICAL DESIGN

Difficulty: Advanced | Estimated cost: \$60–100



**Why recruiters love this:** Understanding CNC from the machine-builder's perspective — not just the operator's — is rare and incredibly valuable. This project proves you understand linear motion systems, stepper motor control, G-code, and mechanical precision — the fundamentals of advanced manufacturing.

Build a small-scale 2-axis (or 3-axis) CNC engraver from scratch that can engrave text and patterns on wood, acrylic, or aluminum. Use linear rails, stepper motors, and an Arduino with GRBL firmware. Design the frame, assemble the motion system, and calibrate it to achieve  $\pm 0.1\text{mm}$  accuracy.

### What you need:

- Arduino Uno + CNC Shield V3
- 2-3x NEMA 17 stepper motors + A4988 drivers
- Linear rails (MGN12) or smooth rods + linear bearings
- GT2 timing belts and pulleys (or lead screws for Z-axis)
- 775 DC spindle motor or Dremel tool

- Aluminum extrusion (2020) or 3D-printed frame
- GRBL firmware (free) + Universal G-code Sender
- 12V 5A+ power supply

### Skills it proves:

- Machine design — frame rigidity, motion systems, backlash compensation
- Stepper motor control and CNC kinematics
- G-code programming and CAM workflows
- Precision measurement and calibration
- Manufacturing process understanding from the builder's perspective

### How to build it:

**Step 1:** Design the frame. Use 2020 aluminum extrusion or a rigid 3D-printed structure. Rigidity is everything — any flex in the frame becomes inaccuracy in your engraving. Model it in CAD and verify joint stiffness.

**Step 2:** Build the motion system. Mount linear rails and stepper motors for X and Y axes. Use GT2 belts for speed or lead screws for precision. Ensure everything is square — measure diagonals and shim if needed.

**Step 3:** Wire and flash the controller. Stack the CNC Shield on the Arduino, plug in A4988 drivers, wire stepper motors. Flash GRBL firmware. Use Universal G-code Sender to test jogging each axis.

**Step 4:** Calibrate steps/mm. Command a 100mm move, measure the actual travel with calipers. Adjust the steps/mm in GRBL settings until commanded distance matches actual distance within  $\pm 0.1\text{mm}$ .

**Step 5:** Mount the spindle and test. Attach the spindle motor to the Z-axis (or use a pen for initial tests). Run a test pattern — a square, a circle, and text. Measure dimensions to verify accuracy.

**Step 6:** Engrave a real project. Design something in Fusion 360 CAM or Easel, generate G-code, and engrave it on wood or acrylic. Document the full workflow: CAD → CAM → G-code → finished part.

### Useful links & resources:

- [GRBL firmware — GitHub](#)
- [Universal G-code Sender](#)
- [CNC build guide — V1 Engineering](#)
- [Fusion 360 CAM for beginners \(YouTube\)](#)



- [Stepper motor basics — SparkFun](#)
- [Linear rail selection guide](#)

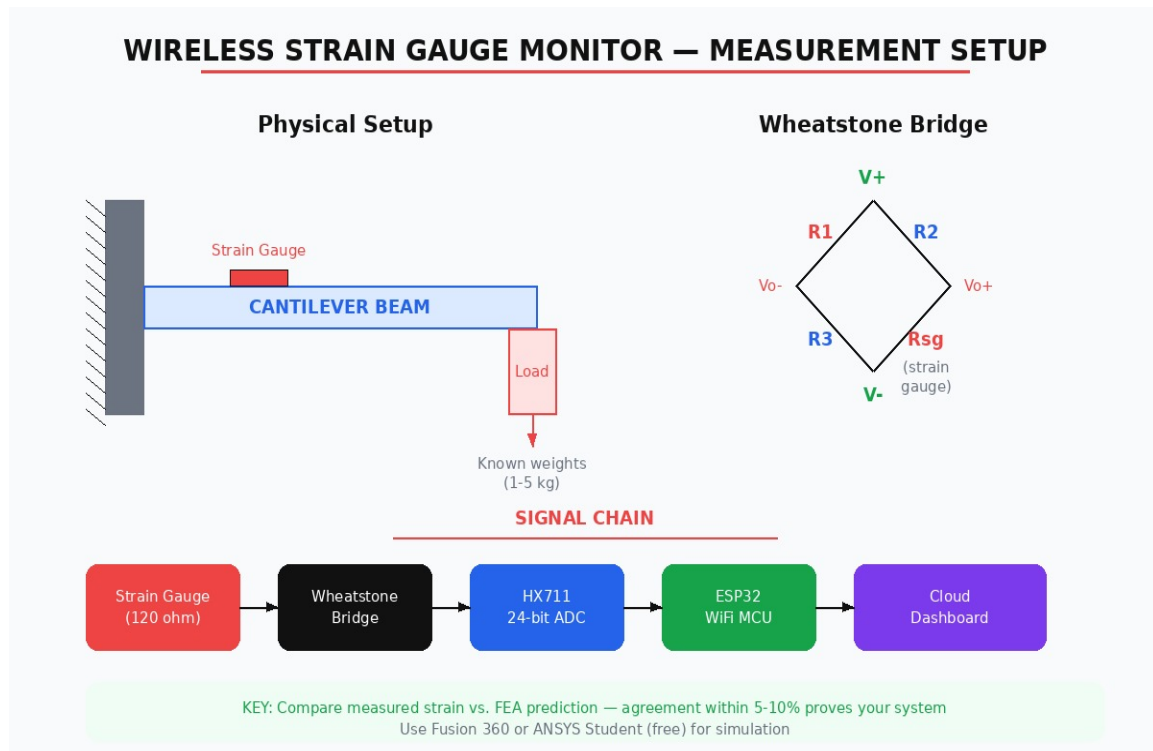
**Level up:** Add a laser diode module (1-5W) as a second tool head for laser engraving/cutting thin materials. Having a machine that can both mechanically engrave and laser cut shows serious manufacturing versatility. Just make sure to add proper safety enclosures and goggles.

# #6

## Wireless Strain Gauge Bridge Monitor

INSTRUMENTATION • STRUCTURAL • IOT

Difficulty: Advanced | Estimated cost: \$45–70



**Why recruiters love this:** Structural health monitoring is a growing field where ME meets IoT. Companies in aerospace, civil infrastructure, and automotive need engineers who understand both the mechanics of deformation AND the electronics of measurement. This project proves you're that engineer.

Build a wireless strain measurement system using strain gauges in a Wheatstone bridge configuration, amplified by an instrumentation amplifier, digitized by an HX711 ADC, and transmitted wirelessly via an ESP32. Apply it to a real structure — a 3D-printed beam under load — and compare measured strain to FEA predictions.

### What you need:

- Foil strain gauges (120 $\Omega$ , pack of 10)
- HX711 24-bit ADC module (same one used for load cells)
- ESP32 for wireless data transmission
- Instrumentation amplifier (INA125 or AD620) — optional for higher precision

- 3D-printed or aluminum cantilever beam (test specimen)
- Cyanoacrylate adhesive (for bonding strain gauges)
- Known weights for calibration (1-5 kg)
- Cloud platform (ThingSpeak or Firebase)

## Skills it proves:

- Strain measurement fundamentals — gauge factor, Wheatstone bridge, signal conditioning
- Analog signal amplification and noise reduction
- Wireless sensor networks and IoT architecture
- FEA validation — comparing simulation to physical measurement
- Data acquisition system design

## How to build it:

**Step 1:** Prepare and bond strain gauges. Surface-prepare the test beam (sand, clean, mark alignment). Apply strain gauges with cyanoacrylate — alignment and adhesion quality directly affect accuracy. This step requires patience.

**Step 2:** Build the Wheatstone bridge circuit. Wire gauges in a half-bridge or full-bridge configuration. Connect to the HX711 ADC. Calibrate the zero point (no load) and verify stability — readings should be rock steady.

**Step 3:** Calibrate with known loads. Apply known weights to your cantilever beam. Record HX711 readings at each load. Create a calibration curve (load vs. digital reading). Calculate sensitivity and linearity.

**Step 4:** Set up wireless transmission. Program the ESP32 to read the HX711, apply the calibration equation, and transmit strain values over WiFi to ThingSpeak or Firebase every second.

**Step 5:** Run the FEA comparison. Model your exact beam geometry in Fusion 360 or ANSYS. Apply the same loads you tested physically. Compare FEA-predicted strain at the gauge location to your measured values. They should agree within 5-10%.

**Step 6:** Build the dashboard and document. Create a real-time web dashboard showing live strain data. Demonstrate the system detecting when load is applied and removed. Document the FEA vs. measurement comparison — this is gold for interviews.

## Useful links & resources:

- [Strain gauge bonding guide — HBM](#)
- [HX711 Arduino library + tutorial](#)

- [Wheatstone bridge explained](#)
- [ANSYS free student license](#)
- [ESP32 WiFi + ThingSpeak tutorial](#)
- [Strain gauge selection guide — Omega](#)

**Level up:** Deploy your system on an actual structure — a footbridge, shelf, or bike frame — and log data over weeks. Add ML-based anomaly detection that flags unexpected strain patterns. This turns a student project into a legitimate structural health monitoring system.

## Quick Reference

| # | Project         | Key Skill Area        | Difficulty   | Cost     |
|---|-----------------|-----------------------|--------------|----------|
| 1 | Robotic Gripper | Prototyping + Sensors | Beginner     | \$25–40  |
| 2 | Wind Tunnel     | Fluids + Data         | Intermediate | \$40–70  |
| 3 | Solar Tracker   | Controls + Energy     | Intermediate | \$35–55  |
| 4 | Thermal System  | Heat Transfer + PID   | Intermediate | \$30–50  |
| 5 | CNC Engraver    | Manufacturing         | Advanced     | \$60–100 |
| 6 | Strain Monitor  | Instrumentation + IoT | Advanced     | \$45–70  |

## What To Do Next

Don't try to build all 6 at once. Pick the one that excites you most, order the parts today, and start this weekend. A finished project beats a perfect plan every time.

When you finish a project, document it properly: clean code on GitHub, a detailed README with photos and diagrams, a short demo video, and your test data. That package is what gets you interviews.

### Essential free software for ME students:

- [Fusion 360](#) — CAD, CAM, simulation, free for students
- [ANSYS Student](#) — FEA and CFD simulation
- [Arduino IDE](#) — embedded programming
- [Python + Matplotlib](#) — data analysis and visualization
- [GitHub](#) — portfolio hosting and version control

**Follow @1nska on Instagram** for more project guides, build logs, and real talk about building as a student. DM me "BUILD" and I'll help you pick the right project for your experience level.

— Enes (@1nska)  
NYU Shanghai, 2026